



Building a GraphQL API with Apollo Server

Course Overview

Course Overview

Hey, everyone. My name is Jonathan Mills, and welcome to my course on Building a GraphQL API with Apollo Server. I'm a multi-cloud architect at World Wide Technology, and I get to architect and build large-scale enterprise applications and APIs built for the cloud. Have you ever built and worked on a REST API and found that over time it became unwieldy to work with where you have to make multiple API calls to get the data back that you need? And then when you did, you end up getting way too much data back and way more than your application needs? And it just results in a little bit of a slow and cumbersome process to interact with your API. Well, in this course, we're going to talk about a different way to approach API design with GraphQL. GraphQL is an open source query language for APIs that allows you to consume multiple resources in a single API call and at the same time allows the client to control the amount of data that comes back from the API so they're not overwhelmed and wasting resources. Now, in this course, we're going to be specifically looking at the Apollo implementation of GraphQL in Node.js. So, what are we going to talk about in this course specifically? Well, first, we're going to define what our data looks like with a schema, which is an added benefit of self-documenting your API. Then, we'll start resolving queries in Apollo in a way that allows you to shape your data that you want to come back from your API. With that, we'll look at nesting data in your query so that you can query across multiple resources in one API call. And last, we'll look at modifying your data with something called a mutation. And at the end of this course, you'll have all the tools you need to build and deploy a GraphQL API utilizing Apollo. Now, before you begin this course, you should be familiar with JavaScript and maybe some basics of back-end API design. But really, that's all. We're going to start from the beginning and build up everything you need to know to get your API put together. I hope you'll join me on this journey to learn how to implement GraphQL with the Building a GraphQL API with Apollo Server course here, at Pluralsight.

Getting Started

Introduction



Hey. I'm John Mills, and welcome to my course on building GraphQL APIs with Apollo and Node.js. Now, there's potentially a lot of different reasons why you may be here watching this course, and so I'm going to spend just a minute or two setting up a specific scenario that you and I are going to use as the framework for this course. And as we work our way through that, I'm going to talk about why you may or may not want to use GraphQL and what it really even is. Now, before we go too far, I'm going to assume you have watched Adhithi's GraphQL: The Big Picture course. Now, it's not really required for this course, and if you haven't watched it, that's okay. But in it, she covers some core concepts that we're just going to jump straight into implementation on. So, if you start to feel like this goes a little fast for you or you feel a little lost with some of the concepts, you may try going back and watching that course. Now, with that in mind, let me introduce you to Globomantics. Now, if you watched my MongoDB course, you may already be familiar with them, and Globomantics runs one of the largest and most awesome technology conferences in the world. And you and I are going to do some API work for them. Now, we're going to pair on this project. And really, that's the best possible way for you to get the most out of this course. So, if at all possible, crack open Visual Studio and code along, and let's get this thing going. Globomantics is building out the Sessions tab for their website. Now, in order to make this happen, they need session data. Now, in the past, they've used RESTful services, and they even still have some in place today, and we'll leverage some of those as we go further along in this course. But for the Sessions page, they found out that they need not only sessions data, but they also need some basic speaker data as well. And with the way their REST services are built, that will result in the browser having to make one call for sessions and then an additional call for each of the speakers in that session data, and that can add up to a lot of calls. Sure, they could change the API to match what's needed on this page. But then, when they start working on the schedule page, well, that needs less session information, and it doesn't need any speaker information at all. So, that results in either too much data and wasted effort for the new custom API, or they create a new custom API for the schedule information as well. Now, you may see where this is going. Either the client just has to deal with what they get, or the back-end team has to create a custom endpoint for each new use case. And neither one of those is ideal, so there's got to be a better way, hence this course, right? Enter GraphQL. The API team has requested that we build a GraphQL API for the Sessions page that will allow the client to customize the data they want returned per query. So, they get to pick and choose the data they want. And they also want it to be able to pull back multiple resources in a single query, so sessions and speakers, etc., etc., kind of like magic. Well, maybe not really. All right, so here we are on the Trello board. Now, there's a lot of stuff over here in the todo list, but let's just start with the basics, and then we can ease into this. So, let's pull the Sessions GraphQL API ticket over to Doing and then assign it back to us. Now, these items here, that's what we're going to be doing for the rest of this module. Sorry, one more thing. We have talked APIs, and we've talked GraphQL. So, now you may be asking yourself, where does Apollo come into this



whole thing? Now, think of it this way. GraphQL is just kind of a framework. It's a set of rules, kind of like SQL or REST. It's not really an implementation. It's just kind of a ruleset around how you interact with an API. Now, Apollo is a Node.js implementation of that GraphQL ruleset, and it does a lot of the heavy lifting for us so we can skip over the rules and just get straight into the business needs. Now, with that in mind, you'll need to have Node.js installed in order for any of this stuff to work. So, if you don't already have it, run out to nodejs.org, and let's get that thing installed. All right, first things first. We need to configure our environment, get our npm packages installed, and set up the basic framework for our application. And then, we're going to create a schema. And in the process of those two things, let's talk about what in the world that this stuff even is.

Building the Environment

In this demo, we're going to get our environment set up and all the npm packages installed and all of that kind of stuff. So let's start by opening Visual Studio Code. Now, do you have to use Visual Studio Code? No. You can use whatever editor you want as we're not really going to use any of the editor features in this course. But VS Code is pretty awesome, and so that's what I'm going to use. All right, let's come in, tick mark to open the terminal, and run `npm init`. And this gets the basic shell of our program set up and creates the `package.json` file right here. All right, next, we `npm install apollo-server`. Now, a quick note about versions. As this course ages, these versions will change. And sometimes that's fine, and then other times things break. So, it may be better to `npm install apollo-server@2.12` just to be sure everything will work as needed in this course. All right, next, we install `nodemon`. Now, `nodemon` is the tool that we use to kind of auto restart our server when we change files. So, if a file changes, `nodemon` picks that up and auto restarts so we don't have to remember to do that. We're also going to use it to build some environmental variables that we'll use throughout this course, maybe set something to production mode or set the port, those types of things. `Nodemon` handles all of that for us. All right, so `npm install nodemon@2.0`. Now we're going to create a new file, `index.js`, and we're going to start pulling all this stuff in. `const ApolloServer = require('apollo-server')`. Now, you see this destructuring syntax with the curly braces quite a bit in this course. But if you're not familiar with it, all it does is go ahead and it pulls just the `ApolloServer` object back from this npm package, and it gives it the variable name `ApolloServer` just automatically for you. And we'll continue to use that syntax kind of throughout the course, so just be aware of what that is and what it's doing. Let's spin this up, `const server = new ApolloServer()`. All right, that gets a new server object set up and ready to go. And then, `server.listen` and our port number. Now, we want to set this up so it's dynamic based upon the environment. But also, we can give it a default of 4000 because we don't have that dynamic stuff wired up yet. And then we're going to add a function that will be called by the server once this starts up. Now it's going to pass an object that contains the URL, kind of that



destructuring thing we were just talking about a second ago. We just want to pull the URL straight out of this object that gets passed in. And this is the URL that the server is running on. So, let's pull that out and write it out to the console. See? That's all it takes. Now we have a functional GraphQL server. Well, functional is a loose term here, but at least it'll run, right? So, let's add our start script over here in package.json. Ready? Start: nodemon index.js. This lets us just type npm start, and it'll spin up our server for us. Okay, now that we have the basics, let's start digging into GraphQL and how it works starting with the schema.

Creating a Schema

All right, I told you that GraphQL allows the client to pick and choose the data they want to get back from the API. Think about it like this. So, a SQL Server query allows someone to understand the shape of the data in a table. And then using that schema, they can pick and choose the parts of the data they wanted to query. So, just like SQL, GraphQL utilizes the concept of the schema for much the same purposes. So, let's write up a schema and then interact with it in the browser. Okay, back in index.js, let's create a new object called typeDefs. Now, this is what will hold the schema we are defining for our API. Now, this takes a little bit of an odd syntax that requires this thing called gql, so we're going to pull that in from Apollo Server as well. Now that we have that, let's start to define what a session looks like. So, we have things like title, description, etc., and each of these things has a type. Now, we have a few different types to choose from in GraphQL. But for now, we're just going to use everything as a string. So, let's add a type to title, and then we'll just kind of finish out this schema. All right, one thing we are missing is an ID, so let's add that up here at the top and give that a different type, the type of ID. One other thing we want to do is denote that the ID field will never be null. You can always count on it to always have a value. We do that with an exclamation point here after the type. So, you can do the same thing with title if we know that we'll always have a value in the title field. All right, so that's the session schema. But one other thing GraphQL requires is a description of what queries are allowed by the API, kind of like a schema for the query. So, let's go up here and add a type of query and denote that we have one query available to us called sessions. Now, the sessions query returns a session, which is the type we defined right down below here. But it doesn't return just one session. It actually returns an array of them. All right, so there we go. Okay, now we have to add this new object typeDefs to our constructor for Apollo Server, but we're out of order here. So, let's move this down here. All right, and then the ApolloServer constructor takes an object, so let's create that. And then, we pass in typeDefs just like this. All right, let's run this with npm start. All right, now we're running on localhost 4000. Let's run out to the browser and see this thing called Apollo Playground that came out of the box with Apollo Server.



The Apollo Playground

All right, now that we've got our server and our schema running, let's take a minute and see what happens on the other side. So, Apollo has this thing built in called the Apollo Playground that runs on the client side for us just automatically. The Apollo Playground is a graphical, interactive, in-browser GraphQL IDE. It's kind of cool, huh? All right, so here we are in our browser at localhost 4000, which is where we told GraphQL to start running in the last clip. Now, what you see is this cool interface that we didn't write at all. Like, we did none of this work. Apollo just created this for us, and this is one of the things I like about Apollo. It's got a lot of stuff that just kind of does for us behind the scenes that we don't even need to worry about. So, one of the things that makes this so cool is if you look over here on the right side, you'll see two tabs, the DOCS and the SCHEMA. So, if we click on the SCHEMA tab, you'll see all the stuff we just defined in the last clip. We have our query. We have our sessions and all the properties that were included in session. Now, how in the world did Playground get all of that information? Like, that's a lot of stuff there. Well, this is done in GraphQL and Apollo automatically through this process called introspection. Now, introspection is the ability to query which resources are available in the current API schema. Given the API via introspection, we can see the queries, the types, the fields, and the directives that that API supports. So, the playground uses introspection to build up all of this information that you see here. And so, if we look at our network traffic for just a second, you can see this query being sent, and it looks ridiculously complicated. And we're just starting this journey, so I don't want you to worry too much about the mechanics here. I want to spend a minute so you can understand the basic concept of what's going on. So, I just want to talk about this for a second. Let's get outside of this Playground interface and use Postman instead so you can kind of see a little bit more of the inner workings of GraphQL. Let's create a new query to localhost 4000. Now, this is a POST, and all interactions with GraphQL API is done over HTTP POST. Now, Postman supports a body type of GraphQL just right out of the box. So, that's what we'll use for these queries that we're about to run. Now, we access the introspection system with a `__Schema` query. And so, let's create something that looks kind of like this. Now, don't worry about the mechanics of this query yet. We'll get there shortly in the next module. But let's just look at the results. Here, we have our query and our session and a whole bunch of other stuff too, so Schema, Type, TypeKind. All these things preceded with the double underscore indicate that they're part of the introspection system, and those just come naturally out of the box. Now, this query doesn't give us everything that you saw kind of over on the playground. But if we dig in a little bit on just one thing, so let's say on sessions, and we run this larger query, we get all of those fields that we defined on the Session schema. Okay, again, don't worry about the mechanics of this query. We'll get to that in the next module. But just understand that with the schema defined, we have this suite of queries available



to us to dig into what our API looks like. Okay, we have a schema, but that doesn't get us too far. We can't actually call the sessions query because it doesn't do anything. So, let's take a minute to recap what we've just done and then look at how to resolve a query by implementing something called a resolver.

Summary

Now, we've gotten just a little bit done so far in this course. But so far, we've laid some important groundwork for what's to come. We started by talking about Globomantics and the task required of us. And in the process, we talked about what GraphQL is and, more importantly, what benefits it provides us. Now, we talked about how GraphQL provides two main benefits to us for this project. The first one is it allows us to control the fields returned back from the API. And secondly, it allows us to pull back multiple resources in a single API call. Now, for the actual work we did in this module, let's run back over to the Trello board. And then, let's open this ticket up that we were working on. Now, we set up our environment with npm packages and the basic shell of an application, so let's check that one off. And then, we added the Sessions Schema, so let's check that off too. Awesome. Let's move this card to Done. Yep, that feels good. Okay, finally, we looked into the idea of introspection and how we have this array of queries that let us look into what's in the API. And that concept allows Apollo to ship this thing called Apollo Playground that we'll use to examine our API and play with our API over the rest of this course. All right, now that we're done with all of that, let's go implement a resolver so that our Sessions API will actually work.

Resolving Queries

Introduction

Now, we have the basic shell of an application, and we've created a schema for our sessions data. Now, we need to make this API actually do some work. So, in this module, we're going to look into this thing called a resolver and how queries really work in Apollo. Now, we'll start by building out a resolver map for our query, and we'll kind of talk about what a resolver map really even is. And then we'll execute our sessions query in the Apollo Playground, that thing we looked at in the last module. And while we're at it, we'll look at some schema directives, things like deprecating fields and stuff like that, and we'll see how those processes work as we play a little bit with our schemas. All right, now that we have a query, we're going to look into organizing our data into something called a data source, and we'll look at what those are. And then once that's done and we've got a data source and we've got a query, we're going to look at the filtering process by



implementing a new query, a byld query, and then allowing basic filtering back up in our normal sessions query. Once all of that's done, we'll have a ton a code written, so we're going to spend just a few minutes on code cleanup and organization. All right, let's get this resolver implemented so we get something that's actually working.

Implementing Resolvers

In order for our API to return data, we need to implement a resolver for our sessions query. In short, a resolver is a function that's responsible for populating the data for a single field in your schema. So, in our case, our schema has a Query type and a Sessions field. So, we need to add a function that tells Apollo how to find those sessions for that Query type. So, with that in mind, the resolver map is just a big object that holds all of those type, field, resolver functions. All right, so let's go grab our Trello ticket and see what we need to do and then add that to the Doing column and assign it to us. So, opening it up, we can see here's the things we need to do. Now, we're just going to be using some dummy data for this. There's a separate card that we don't have to worry about to build out a sessions repository that will later be added into this project. However, since we're not going to cover it here, if you want more insight into wiring up a repository, check out my Using MongoDB course because I build exactly that in that course. It doesn't make sense to replicate that same exact thing over here too. All right, so we need to do a couple things in order to make this happen. We need to add the dummy data to the application, and we need to implement our resolver. So, first and foremost, if you run out to the course materials in this module, you're going to see a data directory with a sessions.json file full of dummy data. So, let's pull that directory into our application. And then, in index.js, let's pull that data in here. And then we're going to create our resolver map object. All right, so for Query and for Sessions, here is our function. And honestly, for right now, let's just return all the sessions. Let's not worry about all the other details associated with this. We'll get to those in a couple of clips. All right, now we need to take this resolver map and add it to our Apollo server constructor. All right, now when we do that, actually a little help here from the IDE. It's resolvers plural, not resolver singular. So, let's go back up here. Let's change this to resolvers so we can still use the object syntax that we want to use, and then we're set. All right, let's save. Now, when we save, nodemon should restart our server automatically. Now, if it doesn't, just run npm start, and you should be good to go. All right, that should be all we need to do to pull session data. So, let's go out to the playground and start looking at our query.

Executing Queries

All right, now that we're resolving queries, let's go over to the playground and see



what this looks like in practice on the query side. All right, so if we click on the SCHEMA tab, we get an idea of what's available to us for a query, and we have a query of sessions, and that returns an array of Sessions, and the type of sessions kind of has these fields. So, if we didn't already know this, if we hadn't written the resolver ourselves, we could just look at this and say, hey, here's the stuff that I can query, and that's super helpful. So, if we come over to the query pane, we could execute all of our stuff right in here. So, I want to start by typing query. And then, query takes an object of all the queries we're going to execute. Now, if you look over in the docs on the schema side, query only has one query available to us called sessions. And so, that's what we're going to type in here, and we're going to execute query sessions. All right, we know we have to actually tell it explicitly what we want to take for our sessions, like what data we want back. You don't just get to do, hey, sessions.* like you would in SQL. You actually have to articulate to it, here are the things we want. So, if we look over here at the type session, we can see what's available to us, and let's take title and room and maybe track. Now, that'll give us some basic schedule information for a session. And then, we'll just go from there. All right, I click the Play button, and you see there it is, just like that. Okay, now notice it comes back inside a data object. So, I've got a single object that's returned from the API. In it, I have a data object that gives me the session information. So, as we grow this out later in the course, we'll see more stuff showing up inside there. Okay, let's change this just a little bit, just so we can kind of see how this works. Let's add in the id and maybe the level, and there you go. Now, we have id and level. It's just that easy with Apollo, we didn't have to write any of that filtering functionality. It just takes care of all that on its own. And it looks at the data it gets back from the data source, from the resolver, and then it looks at the data that's requested from the client and strips everything out that you don't need and then says only the stuff across the wire that the client's asking for. So, that's how queries work in Apollo and GraphQL. We'll go so much deeper into that over the rest of this course. So, it does get a little bit more complicated, but that's the basics. But first, I saw something a little weird on the Trello card, but I want to go back to take a look at that because I think we may need to spend just a couple of minutes talking about some field-level directives.

Field Level Directives

Let's take just a minute and go sideways and talk about schema field-level directives in GraphQL. So, if we look in the Trello card and we open this up, something right here that says Deprecate the track. And you can imagine in an API where the client could just identify which pieces of data they want and you actually kind of set up this contract between the API and the client as, you know, here's the data that's available, and you can't just change that on the fly, right? There's got to be a process for how to make that happen. So, in GraphQL, we have three out-of-the-box directives that we can use, skip, include,



and deprecated. Now, two of them, skip and include, are actually very similar to each other, but just kind of the opposite of each other. So the first one, include, says if a Boolean value is true, it'll include this in the field in our list. So, basically, you say true, and that gets included. Or you don't put include on it at all, and it gets included. But there are times where you would say include false, and you want that to go away. Then, you have skip. Skip is the opposite of include. It's a Boolean value where, if it's true, you're going to skip this one, or you won't call the resolver for that particular field. A good example of that might be if you don't have access to a certain field, you want to do field-level validation or something like that, you can actually exclude certain fields based upon a Boolean of whether or not they should have access to it, something like that. And the last is deprecated, and this is the one we can use to mark a field as deprecated. You should plan on this going away in the future. So, we have @deprecated and then a reason why it's deprecated with the String. And that's how a field would be deprecated and sent to the client as a message that this thing is going away. So, let's take a minute and go back over to our code window and look at how this works. But first, you can do custom directives as well. You're not limited to just these three. But that gets us a little out of the scope of this course, so I'm just going to mention that that exists as an option, and then just kind of leave it there. Okay, back over here in code, let's go to our schema and then go to our track. Now, right after our track, we're going to type @deprecated and then a reason. Now, let's just drop in too many sessions that don't fit into a single track, so we're going to be migrating to a tags-based system in the future. I actually run a conference, and I'm seeing that to be more true with something, a .NET session or a cloud session. Let's just make it tags. Get rid of the track altogether. All right, let's click Save. That'll auto restart. All right, let's go back over to our playground and see how this looks on the other side. Now, right off the bat, we see track has a little squiggly underneath it. So, if we hover over that, we see a deprecation message right there in the IDE. That's awesome. It says, hey, this is going away. But not everybody uses this IDE. As a matter of fact, later I'll show you how to turn this IDE off completely. So, let's go look at how this shows up in the schema if we just query the schema programmatically. So, here in the schema, we get this deprecation message right down here. Well, now we execute it, and you notice you don't get any errors or anything. It just works. I've got all these warnings everywhere in the schema and over here in the IDE, but I don't get an error when I click Play, and that's because deprecation doesn't mean it's broken. It means it's just going to go away eventually. And so, this does not error for anything associated with deprecation. It's just identifying information that shows up on the screen right now. All right, we're just pulling data out of our resolver right now, and you can probably imagine that that's not really the way we want to do that moving forward. So, Apollo makes use of this thing called a data source, and we're going to go implement a simple data source for our session data that later can implement the repository once that thing's written and done.



Implement a Data Source

Let's talk about how to decouple the data interaction with the resolver by using something called an Apollo data source. Now, back over in our code window, we're going to right-click and add a new folder called `datasources` and then a new file called `sessions.js`. Okay, now, the first thing we're going to need in here is the sessions info from `index.js`. So, let's pull that in over here, so `const sessions = require '../data/sessions.json` because it's got to pop up a directory into the data directory. And then, we need to pull in `DataSource` from `apollo-datasource`, so `const DataSource`. There you go. Okay, so that means we need to install that. We don't have `apollo-datasource` yet, so `Ctrl+C`, and then install `apollo-datasource@0.7.0`. That's the version that we're using right now, and so let's stick with that. Now, `DataSource` is just a class, so let's set that up. And it extends `DataSource`, so we'll just pull that in right there. Now real quick, we need to create a constructor. Nothing special there. Just call `super`. And then, we need to create a method called `initialize`. Now, `initialize` gets called with the data source config, including things like caching and context and all of that stuff. We're not going to worry about that part for this course. When we get into a course on caching and things like that, we might look into that. But for the purposes of this course, we don't really need to worry about config. And then last, we need to actually create the methods for getting data out of our data source. So, for now, it's just `getSessions`. All right, last, we need to make sure we export the class, `module.exports = SessionAPI`. And there we go. That's a very simple data source. Now, when the repository is actually created eventually, what they'll do is they'll come in here to our data source and plug that repository in in this class. Okay, now that that's done, let's go over to `index`, and we're going to wire this thing up. So, here we go with `const SessionAPI = require datasources/sessions`. We're going to pull that in. And then we're going to get rid of `sessions` altogether because we don't need that anymore. We pulled it in in the `Session API`. Now, it's over here. Now, we can set up our data source. So, right now, we just have the one, so `const dataSources = a function`. And `SessionAPI` is the only data source we have right now, so `new sessionAPI`. And then we're going to add `dataSources` to our server constructor. Our constructor's got three items in it. It's got `typeDefs`, it's got `resolvers`, and now it's got `dataSources`. Okay, we're done with that. Now, we need to revamp our resolver to use just the data source. So, up here in the resolver, we need add some params to the function. So, what gets passed in up here? We've got `parent`, which we're going to get to later. We've got `args` that's coming up in the next clip. `Context`, we'll deal with that right now. And then `info`, which gives you a lot of information about the query being sent. But really, for our purposes, we're not going to worry about this. So, for `context`, for now, we care about the `DataSource` object only. So, on `context`, let's just pull this out this way, and then we just pull our sessions out of the data source using the `getSessions` method we just created. All right, now let's restart our API with `npm start`. Then, we're going to head over to the browser and make sure everything



still works. We refresh the page, we hit Play, still good. Awesome. Now, let's head over to the Trello board and open up this ticket. Check off each of these items, all four of them, and then slide this ticket to Done. Progress, we're making progress. All right, next, let's start looking at what it means to filter data through the API starting with a byId query.

Filtering by ID

Now that we have a working query and have looked into how to decouple our code into like a data source, let's look at what it would take to implement a byId query. So, if we go out to the Trello board and we see a ticket here for filtering sessions. Let's pull it to the Doing column and then assign it to us. Okay, now, if we open this up, we're going to see two clips worth of work here, a new filter by id query that we're going to do right now and then a more holistic filter on the sessions query that we're going to do next. So, let's go back into VS Code, and let's get this thing moving. Now, the first step in setting up a new query is to add it to the schema. So, right after sessions, we add sessionById. Now, for this query, we're expecting something to be passed into the query. We won't worry about how just yet, but we know something will be coming in. So, now, we have to indicate in the schema what's allowed to be passed in. So, just let the client know, here's what you can pass in. So, we're going to add id:, and then the type is ID. And after that, we return just a single session. Ultimately, that's the whole idea of this query. We could, in theory, just allow filtering by id on the sessions query. But for us, we want a new query that just drops back a single item and not an array of one item. So, there's a decision there, and that's what we're trying to achieve here. All right, now that we have the query, we need to drop down to the resolver map and add a resolver for that query. So, right here, we're going to do sessionById: parent and args. All right, now, for args, Apollo will just send in an object of all the args passed into that query. In this case, we'll only get a single item, id. And so, we can just change this to the destructuring syntax, id. Then, the rest of the args, dataSources, info. Finish that out. Now, what we want to do is something very similar to what we did up here, so the same idea. Return dataSources.sessionAPI. Now, what we want here is a different getter from our data source. Maybe let's just do getSessionById. And then don't forget to pass in the id. Okay, now we need to go create that getter over in the sessions data source. So, let's go right down here below the getSessions function and create a new one called getSessionById. Now, we know we will get an id passed in, so let's add that. All right, now, we just need to filter this list of sessions to just one item by the id. And Lodash has a filter method on it that we can use to make that super easy. So, let's go down and Ctrl+C. Let's get out of our npm start, and we'll run npm install lodash@4.17. That's the version that we're running right now. Okay, now back up to the top, let's pull that in, const _ = require lodash. Okay, now that we have that, let's go back down and filter out our sessions list. Let's const session = _.filter. Now, the filter method takes two params, the collection to the



filter, sessions, and an object to filter on. Now, it could be used with a function here, but we don't really need that, so we're just going to do it this way. Now, here we need to add id, but the id in our data is an int. So, let's pull out the id from the id that's being passed in. All right, and now we return just the first item. All right, let's save that and go down and restart our server. Oh, wait. We need to save index. So, go back over and save. Oh, and it breaks. Let's look at the error here. SessionById is not in the schema. Well, and that's because we have an extra s. All right, so let's fix and save that. Now it works. Okay, let's go back over the query window. If we open the schema, we see now we have two queries, sessions and sessionById. All right, let's think of this scenario. We have a deep link into the session info page that requires all of the session info for a single session, like the title and the description and everything. But also, it will still list like a session schedule over on a side panel. So, we need both the detail info for a single session and this basic info we already have for everything else. So, let's get all that info with just a single HTTP call. Let's come up into the query. We already have everything for sessions. Let's add that up above to our new query, so sessionById. Now, this one expects an input object, so we send id:. Let's just pull this one right here, 84473. Now, let's pull back all the same info. But in addition, we want the description. All right, let's run this and see what we get back. Okay, now, this is a fundamental distinction with GraphQL versus REST. On the REST side, it would've made two calls back to the server, one for sessions and the other for a sessionById. But with GraphQL, I can send both queries over the wire at the same time and just get an array of data with both the queries included. You see it here in the data object, right? See here, sessionById. And if we scroll down a little bit, here's session. Okay, that's filtering to a single item. Now, let's go back and add filtering on sessions.

Filtering by Many Columns

Now that we understand the basics of passing arguments into a GraphQL query, let's implement a larger example by adding filtering by multiple parameters. Now, sessionById allowed only a single parameter to be passed in. Now, the sessions query is going to be a little bit more complicated, mostly because we're allowing filtering on almost any field on Session. So, let's actually pick up everything from Session and copy it up here into the sessions query. Now, we don't need this deprecated thing up here, so let's take that out. Okay, now we've indicated to the client that they are able to filter the sessions query. Now, we just need to make it happen. Luckily, most of the plumbing for that is already done. And if we look at the sessions resolver, you'll see we're already pulling arguments into the session. Like I said before, Apollo will take all the parameters passed on the query and send them through on this args parameter. In the getSessionById resolver, we only needed the id, so we went ahead and pulled that out. But for this one, we're going to keep everything. So, really, all we need to do here is take args and then pass that on to the data source in this getSessions



method. Okay, last piece of this whole thing. Let's run over to the data source and add our filtering logic. So, first things first, let's pull in args into the `getSessions` function. I know really we can just copy what we did down in `getSessionById`. It's just a simple call out to `Lodash` and `filter`, and it just works. Now, I'm not going to worry about the implementation here because really this is where the database logic would go. And if you want to work through that part, you can check out my [MongoDB course](#) or my [Express course](#) where I covered all of that stuff. The process is exactly the same. All right, let's head over to our playground and add a filter to our sessions query. All right, room: Europa and track: .NET. Okay, now, this red bar here indicates that we may run into trouble. But we're going to click the Play button anyway so you can see what errors look like when they happen. All right, now, instead of data, now we have some errors. And we actually have two of them. So, let's collapse some of this so you can see them. Looks like on the import, both the track and the id are required, and that's not right. But it's a good way to show that the exclamation point syntax works in both directions and give you a good intro to how errors work and kind of what they look like. But now, in a couple of modules, we're going to dig into errors quite a bit. But this is a good start just to kind of see how errors work and how Apollo sends things back. Okay, so let's run back to `index.js` and remove the exclamation points here. There we go. The first item is still `getSessionsById`, and the second group is only Europa and .NET. Excellent. Nice. One last thing before we go. The code and indexes, it's getting a little long. So, before we go to the next module and start adding in speakers, let's look at how to pull this apart into something a little bit easier to deal with.

Cleaning Our Code

All right, as we've built this out, our `index.js` file has grown just a little bit. And I want to be a little proactive before we start adding in all the speaker stuff in this next module. So, let's take a minute and start pulling apart our `index.js` code into some meaningful files. Honestly, there's a lot of ways to do this. And instead of covering a whole bunch of different options, I'm just going to show you a pretty easy and straightforward way to do it and leave that as a jumping off point so you can go do some research and find the way that works best for you. Now, at this point, with the limited amount of code we have written, it's a pretty easy process. So, let's start with our schema. So, right here, let's cut the schema and change this to `const typeDefs =` and just `require('schema.js')`. All right, now we've got to right-click and create a new file, `schema.js`. And just `module.exports =` and then paste everything we just cut out of `index`. It's just that easy. Well, almost. We do need the `gql` thing. So, let's go up and pull that in, `const gql = require('apollo-server')`. All right, that's it for the schema. Let's go and cut out our resolver map, and we're going to change that to `const resolvers = require('/resolvers')`. And so, that way, now we're pulling in the same thing just from the `resolvers` file, just like we did with `schema`. Now, let's create a new file and call it



resolvers.js, and we're going to add `module.exports =` and paste that into our new resolvers file. And now we're done. Let's go over to the playground and check this out. So, hit Play. It still works. Nice. Okay, you may be thinking to yourself, I don't really imagine a production website wanting this playground available to the general end user. And there will probably be scenarios where introspection isn't even really wanted. And well, that's easy enough to fix. Let's look at that really quick. So, if we go to `index.js` and look at where we create our Apollo server, we see the constructor takes an object, and there are two things we can pass in here. So, let's start with `introspection: false`. Now, if we go over to the playground and look at the schema, you see it no longer works. You could still run the playground and refresh the page, but the schema and docs no longer work because we've disabled introspection. We can also go into `index.js` and go back and do `playground: false`. And now the whole thing's missing. Okay, one last option for this. Let's pull these last two things we typed out. Refresh the page, so you see it working again. Now, go into our `package.json` file. And since we're using `nodemon`, we can add this thing called `env` variables. So, let's add a new `nodemon` config `env` our `NODE_ENV`, and we'll set it to `production`. Now, for this to take, we do need to stop and restart our server. And so, let's go to the browser and see we've got no playground. Now, this does leave introspection in place. Production mode only disables the playground, but that's generally the production style that we would want. Okay, let's take that back out so we can use the playground for the rest of the things to work here. Let's go back over. Oh, doesn't work because, even though it restarted, we have to restart the server completely so it picks up the new node config. So, let's stop and restart it. There you have it. Click Play. Everything still works. All right, let's run a quick summary on what we have talked about over this last module, and then let's go add some speakers into the mix.

Summary

All right, we did quite a bit actually in this module, and we are well on our way to building out a full-blown GraphQL API for our Globomantics website. We started with a conversation about resolver maps, and we didn't go too deep. We're going to go deeper into resolver maps in the next module, but we kind of just started with here's what a resolver looks like. And then we started executing queries. We talked about how to pull up the playground and query something and filter and all of that. And then we had schema directives. I talked a little bit about how to deprecate something, and we did skip and we covered the `@` sign notation of a schema directive. We implemented the data source. We had to pull data from somewhere, right? So, we implemented a very simple data source, in this case. We'll do more with data sources in the next module, but we kind of just started this idea of implementing a data source. And then, we completed our tasks. We started filtering queries with `query byId`, and then we actually let you filter by passing in additional items into just the general sessions query to filter



that down. And then, we had some good cleanup. We cleaned everything up and broke things apart, and I kind of started you down that path of how to break things up into separate files as we go. All right, so real quick, let's go back over to our Trello board, and here we had our task to filter our queries. Let's open that up, and let's click both buttons. We did both of those things. All right, let's drag that to Done. Awesome. Now, if we look over here on the left sign, you see next we need to start introducing speakers to our list. So, up next, let's go add speakers and start pulling data from a separate RESTful web service.

Nesting Data

Introduction

All right, so we've got a working API that returns sessions for the Globomantics website, so that's done. Now, let's dig in a little deeper and start to explore the real power of GraphQL. So, if we go to the Trello board where we're following along here, we have a ticket to add speakers to our API. And this is really where the power of GraphQL kind of shows up. So, let's drag this ticket to the Doing column and assign it to us so we can go get it done. And let's talk about what this means and how that's going to work. Okay, so sessions have speakers, and speakers have sessions. It goes both ways. In the REST world, you'd have two separate endpoints based upon a specific resource. You'd have a sessions endpoint, and you'd have a speakers endpoint. Well, in the GraphQL world, we maintain a single endpoint and use the query to direct the data that comes back. So, we kind of pick and choose what we want to come back. So, in this module, we're going to build out a speakers query that's going to look very similar to how the sessions one looks. Now Globomantics already has an existing Speakers REST API that they want to maintain and not rebuild the logic for, and you'll actually see this quite a bit out in the real world. We have all of these REST services that exist, and we don't want to rewrite all that back-end logic just to take advantage of GraphQL. And so, what we do is we build a GraphQL endpoint that kind of sits on top of that REST service. And so, we're going to talk about how that works and what that looks like. So, we're going to use something called the rest-datasource to pull that data back. Now, that'll get us a speaker list, but it won't help with listing the speakers inside of a session. So, we're going to talk more about resolver maps and how to build out logic that says, hey, given a specific session, give me a specific speaker. So, let's get started by getting our Speaker API up and running.

Speaker API

All right, in order to build at a REST data source for API, we need a REST data



source to connect to. And so, in production, they're going to point it to their production speakers API. But for us, we don't have one, so we kind of have to mock something up. So, luckily, I've provided a mock speaker API that you can use to build this out. So, if you head out to the resource files that are included in this course, you're going to find a speakers directory that has an API in it that you can use. So, if you open up a terminal window in that directory and run a quick npm install, that will get you all set up. And then do an npm start to get it up and running. Now, you can make sure this is working by heading out to a web browser and just go to localhost:3000/speakers, and that'll get you a speaker list just like that. All right, if you have that data, if you see data here, leave this up and running, you're all set, and you're ready to work through this module. Make sure you leave your terminal window open with localhost:3000 up and running because we're going to hit that over and over again as we go through this module. All right, let's go get our speaker added out to our GraphQL API.

Speakers Schema

So, on the surface, adding the speaker is going to be very similar to the sessions query. We're going to end up typing a lot of the same code we just had over on the sessions side. So, let's come over here to our schema.js file and see what we want to do here. All right, so right here under the sessionById, we're going to want to add in two speaker queries. Speakers is going to return an array of speakers, so just like sessions. And speakerById returns a single speaker. Again, just like the sessions one did. Well, now, right below that, let's define what a speaker looks like. So, let's type Speaker, the type of Speaker, has an id that's required, so exclamation point. Then, we're going to do bio, which is the String, name, which is the String, and sessions, which is an array of Sessions. Excellent. Okay, now, let's pop over to the resolvers file and fill that in as well. All right, we need to add our two query resolvers right here in this resolvers file below sessionById. And they look very similar to the ones that are up above. So, speakers takes parent, args, dataSources, and info, and then it's going to return dataSources.speakerAPI. Now, we haven't created that yet, but we're going to create that in a minute, .getSpeakers. Excellent. And then, speakersById, same thing, parent, we pull out the id, dataSources info, and then we're going to return dataSources.speakerAPI.getSpeakerById, and we're going to pass in the id. Excellent. All right, now we need to create this speakerAPI data source. And, as I mentioned before, we need to leverage an existing REST API for the speakers, and we could write all the code to do that ourselves. But let's look at the built-in REST data source that Apollo provides that kind of works exactly for this use case.

Working with Rest APIs



Now, laying a GraphQL wrapper over top of an existing REST service is a very common use case that I see. And so, fortunately, Apollo comes out of the box with a REST data source that handles most of the heavy lifting around REST APIs for us. All right, let's dig in here and create a new file in datasources. So, just right-click, New File in datasources called speakers.js. All right, now, this is the file where we're going to wire up the REST data source into our application. Just kind of like when we were over here in sessions.js, we would put a Mongo or a SQL Server or whatever inside here that would go pull all the stuff from the database. Well, in speakers.js, we're going to build that same kind of idea, but around a REST service. So, we've got the speakers REST service; now we just want to pull that data in. So, the first thing we need to do is pull in something called apollo-datasource-rest. And so, what we're going to do is just open up a new terminal window and go `npm install apollo-datasource-rest 0.9.3`, and we're going to run that. All right, now, let's pull that in. So, `const data source = require apollo-datasource-rest`, and there we go. All right, now, just similar to what we did in sessions, so this is going to be a class, `SpeakerAPI`, extends `RESTDataSource`. So, we build our constructo, which is all pretty straightforward. Except now, we get to the one thing we're going to add in here. So, it's where it separates itself a little bit from sessions. Over here in sessions, see, we just have the constructor super, and we were done. We didn't really do anything with anything in here. But in speakers.js, now we get to start doing stuff, so `this.baseURL = localhost:3000/speakers`. There's our constructor. All right, now we're going to add two methods. Now, in sessions, we were just returning a `Lodash` filter in both, so there wasn't really anything fancy going on. Now, though, we're going to pull from a REST data source, which is an asynchronous thing. So, we want to do `getSpeakerByld`, and then we want a pull from a `get` method, so `const data = this.get`, and then we want to send in `/id`. But as you know, that's not kind of how this works. So, `this.get` is going to return a promise, and we don't like dealing with promises, so we're just going to use `await` here. Now, in order to use `await` here, this whole thing has to be asynchronous. And if you are unfamiliar with `async/await` in JavaScript, there's some other courses, none of them are mine, but there's some other courses out there that kind of talk about what `async/await` means and how that works. So, basically, at its core, we can just `await`. Instead of doing a `.then` like a promise, we just wait for the results to come back. And you can only use the `await` keyword inside an `async` function. And luckily for us, the Apollo data source and all that allows us to have an `async` function, `getSpeakerByld` that lets us do this. And then, we need to return data. And then, we're going to do the same thing, `async data = await`. Oh, that doesn't make any sense, does it? `Async getSpeakers`. There we go. And then return data. All right, that's it for the constructor. Now, we do want to go in and make sure `module.exports = SpeakerAPI`. All right, that's our `SpeakerAPI`. That's speakers.js That's it. It's this thin wrapper around a speakers data source. All right, now to wire that up, we've got to go back over to index. And then, up here where we pull in the `SessionAPI`, we need to pull in the `SpeakerAPI`. And then, down here in `dataSources`, we need to create it. All right,



that's all we need to do. Now, if I recall, I think I typed something wrong just a second ago. Let's go back and look at our resolver. Here we go, `dataSources.speakerAPI`. All right, now, if I come back over to our previous thing, we'll see we restarted. We changed our stuff. So, let's open up our browser, and everything's working. But now, if I go to my schema, we'll see I have a `speakers` and `speakersById` query. So, let's try that one out. And again, I love GraphQL because I can keep these other queries I have in here, and we're going to keep using them. But here, I'm going to say `speakers`, and we're just going to pull back the name. That's all we're pulling back for right now. We're going to run that. Oh, `data` is not defined in `getSpeakers`. So, again, errors coming back. Make sure you pay attention to these. We're going to play with those here in just a little bit because in my `SpeakersAPI`, there we go. So, I got my `speakers`, and there's a lot of them, and then we get to our `sessionById` query, then we get to our `sessions` query. Everything's working. That's awesome. So, next, we're going to work on nesting our resolver. So, it's one thing to say, hey, I've got `speakers` and I've got `sessions`, but I want to embed `speakers` and `sessions` together. And so, let's work on doing that next.

Nesting Resolvers

All right, so now that we've got `speakers`, and we've got `sessions` separately, now we need to learn how to use them together because the requirement we have is we want `sessions` to pull `speakers`. Let's take a minute and look at how to nest to these resolvers. So, here in our resolvers file, you see right now we've got query, and that's the only section we have. And so, I've described this thing as a resolver map because it maps things together. But right now, all we're mapping is functions to a query, and that's simplistic. So, what we're going to do now is we're going to actually start building out the REST of the map so that you can see how this really works. So, the first thing we're going to do is we're going to come down here in `Query`, and at the end of `Query` see it. So, at the end, after that, now we're going to start building out resolvers for `Session`. So, we've got the query resolvers and actually, see, it maps back to schema. So, if you go to your schema, we've got our type of query, and then we have our resolvers for `sessions`, `sessionById`, `speakers`, and `speakerById`. Now, we've got type `Session`, and we're going to build resolvers for `speakers` because it needs to go resolve how to look that up. So, this is what we're building is the resolver for type `Session` in for `speakers`. All right, so if we come back over here and now we say `Session` and then we say `speakers`. So, up here, notice it says `parent` in all of them, and `parent`, for a query, doesn't make sense for what we're talking about. But for a `speaker` in the `session` type, the `session` makes sense. So, `parent`, in this case, is the `session` that's being passed in, and so we'll use that now. We didn't use it before up in `Query`, but now we're going to use it as `Session` when we start filtering our `speaker` list. All right, `args` is still `args`, we're not passing any of those yet, and then our data sources. All right, now we need to go and get our `speakers` just the same way we did before. If



we look up here at our speakers, so here we have `dataSources.speakersAPI.getSpeakers`, we're going to do that same thing. So, here, we're going to do `const speakers = our getSpeakers`. But wait, this is await. We need to await this, which means speakers needs to be async. You can async the whole function attached to a resolver just like that. All right, now we need to filter this thing, same way we've done before in other places over in our data sources. I'm just going to say `const returns`. So, basically, what we want to do is we pull all of the speakers out of the API, and then we're going to filter those speakers down to just the speakers that exist in the `session.speakers` list. It could be all of them. It could be one of them. It could be, I don't know. If you've got a panel, there could be a ton. So, `speakers.filter`, and we pass into that a speaker and then a function that returns true or false whether or not that speaker exists in the list that we want. So, we're going to do `lodash.filter`, and I know we don't have Lodash. So, let's come up here and say `const _ = require lodash` just like that. All right, now back down here, `return _.filter session.speakers`. So, we're going to take the list of speakers in that session and say, hey, if in that list of speakers we see the `speaker.id`, then give me a list. So, this is looking at the array of speakers in a session. And for every time I see an id in my speakers id, it's going to return true. But I want true or false, so `.length` is greater than 0. So, it's going to return me an array. It's either going to be an empty array. It's going to be a array of one item. If it's a length of greater than 0, keep it. And so, basically, this is going to build a list called `returns` of the speakers that are listed in `session.speaker`, and we're going to return that list. Now, we could have just looped over and gone and gotten our list for our speaker for each one. Depending on your data size, that's really what matters. So, in this case, I just wanted to pull back the whole list cause it's a very small list. I'll just pull the list back, and I'll filter it here. That way, I'm still just making one HTTP call. If you've got a large list, you could loop over speakers and make multiple calls, but this is a little bit easier way. Also, and I'll show you this here in just a second, the REST API caches. And so, let's run this, we'll save this, and I'll show you what I mean when I say it caches. So, let's open up our browser. And then, down here, in `sessionById`, let's add speakers and name and then run this. Now, we say we have speakers, and we have `sessionById` that has our speaker in it just like that. Now, one thing I want to look at and I want you to see here is this is our terminal window for our REST API. So. we've got `/speakers/speakers/speakers`. I'm going to clear this up a little bit, and I'm going to run this again. And I want to show you how the rest-datasource works. So, in theory, I run this query here to go get all the speakers and their names. And then, again, I run that same get command again to go get the speakers for this session id. And so, when I run this, really I'm making that API called twice. And notice, I only made it once. And actually, if I come down here and I do the same thing, now we're just adding speakers to every session that's in Europa. And you see, there it is over and over and over again. I still only made one call to `getSpeakers`. And so, when we come back over, every time you make this call right here, the rest-datasource caches that call for the duration of the API call. And so, if I click Play again, it's not still cached. I have to go and pull it again. But it caches it so that I can do



something like this inside session, and I don't have to worry about making repeated calls to my REST API over and over again. So, that's a big advantage of using the rest-datasource. Okay, now, look at this. We've got our query. We've got our session. If we added speaker, I'm not going to do speaker here for this. But if we added speakers, that would be another thing, and then we've got mutations coming up, which is going to be another one. This resolvers file starts to get kind of wonky and large. So, let's take a minute and talk about how to modularize that resolvers file so you can pull it apart and keep it maintainable.

Modularizing Resolvers

All right, so we have got speakers and sessions nested, so now you can pull speakers for a given session. Let's take a look at how to break this up a little bit. And so, as you've got sessions and speakers and mutations and other things, this file starts to get big, and let me show you what I mean. We've got Query, and we've got four queries that can be executed, so we have four methods associate it with it. And we've got Session and speakers associated with sessions. And then, in theory, we would also have speakers that would have sessions associated with it. And then, we're going to do mutations in the next module, so we'll have a Mutation section. And there's all of these things that are going to start getting put into our resolver. And really, the general rule of thumb is you don't want all of this packed into one file because that means if you want to add a new resolver to Session, you've got to crack this file open, and somebody else is adding a resolver to Speaker. They're dealing in the same file, and that gets kind of gross. And so, what we want to do is break this up up into multiple files so that we can deal with things in a little bit more modular way. And so what we're going to do in order to make that happen is we're going to just right-click right here, create a new folder called resolvers, and we're just going to pull Query and Session out to their own files. And so, if we right-click on resolvers and create a new file called query, then right in here, we can just do `module.exports =`, back to the resolver, and just pull this, just like that. Make sure I got the closing brace, save. It's just that easy. That's what we need. Now, let's leave Query right there. And so what we do is we pull Query in, so `const require resolvers/` query. There we go. And now, we do the same thing for Session. So, let's create a new file called session. Same thing. `Module.exports`, and we come over to resolver. Let's pull that out just like that and save. Now we need `Lodash`, so let's pull that out and drop it there just like that. That's all it took, just that fast. Okay, so let's don't forget we've got to pull Session in. Now, let's try this out. It should just work, right? Everything still works. Refresh the page. Run it. All right, we're in good shape. So, let's take just a couple of minutes to summarize what we have here and then go and talk about how to modify data. We've talked about how to get data. We'll talk about how to modify data with mutations.



Summary

All right, so we've come quite a ways on our journey through building out this GraphQL API for our speakers and sessions list for Globomantics. Now, in this module, we allowed the speaker list for sessions. So, basically, we went in, and if you pull a sessions query, you wanted to see the speakers associated with that session. And in order to do that, we had to build a speaker query. So, we built the schema, we built the resolvers, we built all of that out. Now, we reused an existing REST API, and so we used the rest-datasource object that comes built in with Apollo, so that was super helpful in order to deal with all of the the REST API calls and the caching associated with using that API. We also nested the resolver map. So, we talked about how to build a resolver for speakers associated with sessions, and we kind of worked through that resolver map. And then, we cleaned up our resolver a little bit as we went as well. Now, real quick, let's head out to our Trello board, and let's pull this Speakers to Sessions over to Done. That's done. But if we look over here on the left, you will see two cards. We have a Favorite Session and an Add New Session card. And so, those are two cards we're going to pull over in the in the next module. We need to create something new, and we need to edit something. So, in this next module, let's look at this thing called mutations and how, in GraphQL, we mutate data. Instead of updating it or adding it, we just call it mutating. So, let's look at mutating data in Apollo.

Mutating Data

Introduction

All right, so we've come quite a ways in our journey of building out this GraphQL API with Apollo. And we've got a nice API built honestly. We've got a sessions query that lets us pull back session and speaker information, and we're filtering, and we're doing some other things. But at some point, we do need to modify data. I mean, you can't just query. Oftentimes, on an API, you want to add some things or modify data or remove things. And so, in this module, we're going to work through what that process looks like. So, a couple of things we're going to work on. We're going to toggle our favorite session. So, I'm going to show you how to update data by taking a session, and then we're going to just mark it favorite or not favorite. And then, we're going to add something new. So, we'll add a completely new session, and so that's another thing that we can work through. And then, we'll spend just a few minutes on enums, and I'll kind of show you what they are. I don't like to use them in most of the things I'm working on, but there are some use cases for them. So, I'll show you what they look like and kind of how you would go about using enums in your code. All right, so let's get started by doing data modification in Apollo and GraphQL.



Modifying Data

All right, the first thing we're going to do in our journey to learn how to modify data or mutate data as we call it in GraphQL is to talk about modifying data. We're just going to update some data and see how that goes and kind of work through the mechanics of how this works. So, if we look at our Trello board, we have a card to favorite a session. And basically, what we want to do is allow a user to favorite a session. Now, we don't have users yet in this API, and that's actually going to come in a separate course. We want to allow that user to favorite a session. So, we're just going to build a toggle on our current API to allow that to happen. So let's add ourselves to it and then get to work. So, basically, what we want to do in our schema.js file is come down to our session. And right here, we're going to add favorite, and it's going to be a Boolean. And if we come back to our playground, here in this first one, right after title, let's add favorite there, and we should get null because it doesn't exist. We haven't built that yet. But what I want to now do is allow a mutation that does this. Okay, so the way we go about doing that is notice in here we have a type of query. We have a type of speaker and a type of session. Those are the types associated with our schema. In order to modify data or mutate data as we call it in GraphQL, we just add another type of mutation. And this type of mutation actually works very similarly to how Query works. If you look at Query, we basically have four methods or four things you can do to query data. And the expectation is we're just here a pull data out for a query. For a mutation, it's the same idea where we're going to create a method called toggleFavoriteSession, and that's going to be a mutation that we can do on this data. Now, in REST, we had PATCH and PUT, and those were the options, and then POST to create new data. In GraphQL, we just have this one idea of mutation that allows a lot more fine-grain control over the types of things that you're allowed to do. So, we're going to pass in an id. So, basically, you're just going to pass the id of the session into this mutation, and it's going to just take whatever the current favorite flag is and toggle that to whatever it's not. And we're going to pass back a Session. It's just that easy. All right, in order to make this work, we now have to go into our resolvers. So, let's go to resolvers.js. And notice, it's the same thing we have. We've got Query and Session. Just like our schema had Query, Speaker, and then Sessions, now we have Mutation. So the same thing here, we need to add a mutation type to our resolver. So, that means now we've got to create this file. So, let's right-click on resolvers, create a new file, mutation.js, and we're going to do the same thing like we would in a query. So, if we go to query, we do `module.exports =`. And then, we're going to the same thing that we do with `sessionByld`. We've got `parent`, `id`, `dataSources`, and `info`, and we want to return something. So, we're just going to pull that over there. Now, this is now `toggleFavoriteSession`. That's what we put in our schema, `toggleFavoriteSession`. And then, we want to return our data source method. We're not going to do the work in our resolver. We're just going to pass that off because we don't necessarily know how that works in this resolver. The



data source is going to be responsible for fixing that. So, we'll just let the data source take care of it. So, if we go one more step to our data source, to our sessions.js, let's just copy this and change this to toggleFavoriteSession. And the reason we just copied it is because this next line's going to be exactly the same. We're just pulling the session out of the list of sessions, and then we're going to return that. But before we return it, we need to toggle the favorite session. So, session.favorite = !. If it's true, it'll be false. If it's null or false, we'll flip that to true, and then we return it. So, let's save that. Come back here. Module.exports, that still works. Save that. And then, our resolvers, we just need to pull that in. So, we should have restarted. Make sure that you're still running down on the bottom. And then, let's pull our playground up. Now, favorite is there, let's run this, and it still says null. Excellent. Okay, now in order to execute a mutation, notice we have query that we've been running this whole time. Now, we also want to do mutations. And so it's a separate thing. So you say if we add mutation and then toggleFavoriteSession, we're going to pass in, let's pass in the same id of 84473. Just the same one we're messing with down below. And then, it's going to return a session, so we do the same thing. Let's just do title and favorite just to validate favorite. Okay, now notice something. I've got two red marks over there on the left side of the screen. And the reason why it's that way is because query and mutation can't survive together. You can't do both of those things. Whereas, down here, you've got query doing a whole bunch of stuff. Mutation we're going to pull out. I'm actually doing a separate tab just like that. So, we're going to run this. And now notice, I've got favorite of true for the Secure Programming for the Enterprise session. So, it flipped the favorite to true and returned it back. Now, if I go to speakers, this query here and run this, so our speaker list is still there. Now, notice favorite is true. So, we are now modifying data. And instead of opening up session to say, hey, you can change anything, right now you can only change this one very specific field on session, and that's something that's a little bit unique to how GraphQL does it versus REST. So, it's very fine-grained, and you have great control over what you allow and what you don't allow just by having the specific functions basically that you can call through your API. Now there, we just passed that id in. But if we want to create a new session, then things become a little bit more complicated because now we have to talk about this idea that a session that you pass in is different potentially than a session that comes back out, like we might have id or modify date or things like that. So, let's talk a little bit about input types in this next clip.

Input Types

All right, the next task we want to do is to create a new session. And before we implement that, let's take just a minute and talk about this new concept called an input type. All right, so if we look at our schema.js file down here in mutations, we basically want to add an addNewSession mutation that allows the user to create a new session. And up here in sessions, notice we passed in all of these things, id,



title, description, startsAt, all of that kind of stuff. And we could do that here, but that becomes a little bit cumbersome, and it's not really declarative of kind of what we're trying to do. And so, what I really want to do is just basically allow them to pass in a session just like that, that is of type session. Now, in order to do that, it actually isn't really a session. If you look at what session looks like, session takes an id. And we don't necessarily want them passing the id in. So, we don't really want session with an id; we want session without an id. And if you had modDate or other things that are kind of server-defined, you don't want them passing those things in. So, we don't really want a session; we want a slightly different type than what session really is. And so, we actually do that by declaring an input type, so you actually say input. And so, we'll just call it sessionInput, and we can actually copy— Let's just do this. Let's just copy these things for right now as the input values for our createNewSession mutation function. And so, basically, what we're now saying is if we save this, it actually breaks because, and if we look at our error, must be input type, not session. So, we don't want it to break there, so hold on. Instead of Session, we need to do SessionInput. Now, let's save it, and we're running again. Okay, but if we come over here to our schema and we look at addNewSession, it actually says we take a SessionInput type. If we make this a little bigger, we can actually just say hey, look, this is what's required to pass into that createNewSession method that we have on our mutation. If you look at our docs, even here's our addNewSession with everything involved, what it takes, what it returns, all of that. Okay, so now we have our input type defined. Let's actually go and implement this mutation just so you can see how easy this is.

Adding Sessions

All right, now that we've got our input type defined, let's just finish out this mutation of adding new sessions. All right, so in the last clip, we built this mutation, addNewSession, and we created an input type called SessionInput. That's two of the pieces we need in order to get this new mutation to add a session up and running. Now, there's two other pieces that we need to do real quick, and this should actually be pretty fast. Let's take this addNewSession and go up to mutation, and let's add that there right under toggleFavoriteSession. We've got the same idea here, parent, args, dataSources, and info. Now, what we want to do is basically pass those args off to our data sources, just like we are everywhere else. So, return dataSources.sessionAPI.addSession, and then we just pass the args in. Now, those args are going to be the session, and I'll show you this. Before we implement it, I just want to kind of show you what that looks like. So, if we go to our datasources, to sessions, we need to create this new addSession. And let's just console.log that because basically what we want is we want the session passed in. So, as we save this, and then we save the mutation. Let's make sure we're running. Yep. All right, let's go over to our playground and see how this ends up being passed



through with this input type. So, let's do mutation, `addNewSession`. We're going to pass in a session, and we're going to pass in a title. And what else do we need? If we look at our schema, if we look at an input session, we need a title and description, `startsAt`, all of that kind of stuff. I don't care about most of this. Let's just add the title and then description. All right, that's the add. Now, we have to have passed back what we want to see. So, let's add the title and the description. Okay, so let's run this. Notice, we get nothing back, which we expected because we haven't implemented it yet. But hopefully, we will see something in our console. And what we see in our console is this. So, we have `args`. The object is the outer curly braces, and we actually have session, the object session, as part of that `args` list. So, really, all we have to do is if we stay in mutation, instead of `args` here, notice up in `toggleFavoriteSession` we pull `id` out. We can do the same thing here. We can actually pull out session and then pass session through. Now, when we run this, it still doesn't work. But notice, now we just have the basic idea. Okay, so what we want to do in our sessions data source, in our `addSessions`, we actually just want to do a `sessions.push` session. And, in theory, your database or wherever you're saving this would auto generate an id based on that. But just for the sake of right now, let's just say `session.id = 12`. And then, we're going to return session, just like that. So, now, when I run this `addNewSession`, we're going to want our id. There you go. We've got our `addNewSession`, id, title, and description. And if we come over to our query, and instead we want to just pull 12, that's the one we just created, notice we've got our new session. We took bio out and our id. So, that's working. We now are successfully creating a new session when we run this mutation, and that's kind of the idea. And so, instead of the whole CRUD mentality around REST, in GraphQL, we've got basically a methodized way of modifying data in Apollo. So, we can modify data with `toggleFavoriteSession`, or we can add data with `addSession`. And then, as you can imagine, if you wanted to allow them to edit all of session, then you could do an edit session. Or you want to delete session, you can do a delete session. It's just you can create a method endpoint for whatever type of thing you want to do. All right, let's take a quick look at something called enums, which is another schema type that lets you dictate what is allowed to be passed in on a given field.

Enums

All right, the last thing we're going to look at in this module is enums. And we're going to talk about them, and I'll kind of show you what they are cause on their surface they seem pretty cool until you kind of get under the covers and you notice that they're not necessarily a tool that you'll reach for very often. So, if you look here at sessions, we have a room type that's a String, and we have a limited number of rooms. Actually, if you go into our sessions data, you'll see here we've got Sol, and here we've got Jupiter, and up here we've got Saturn. And in our query that we already have built, we're looking for Europa. So, we've got just a



few options of what the room can actually be. And so, wouldn't be kind of nice if we could let the end user know, the client know, what options they have when they're going to use room in their query. And so, the way we do that is with a thing called an enum. And so, you come down here into your schema and just type enum of Room. And then, we can actually just type the options. So, you've got Europa, Sol, Saturn. Let's just say that's it for right now. We don't have to type them all out. And then Room now becomes type Room, now the input type of room. So, in our sessions list, the things that we can pass into a session, room becomes the enum Room. So, let's save that. Then, if we come over here to our playground and we refresh, now notice we've got an error right here, and that's because argument room is invalid. What that gives us is we can actually say room, and then it lets us know what the options are, so Europa. If you come over into our schema, you actually can see, hey, here's the options for Room. And so, now, we can run this just with that. Let's get rid of this speakers one. And now you'll notice everything's just Europa. And so, that's kind of cool. It just gives you an option to kind of wrap that up a little bit. Now, for something like this, those rooms will change. I don't necessarily want to change code every time I add rooms. And so, it's not really the right place for this. There's also one other odd idiosyncrasy about this, which is why I don't recommend enums unless you really specifically know what you need to do. The convention for enums is kind of like a list of all capital letters. So, we want EUROPA and SOL like that and SATURN. So, we actually call out the fact that these are big things. And so, but that doesn't translate. So, if I run this now, we want to change this to Europa like that, notice we get nothing, and that's because they're not all capitals. Over in our data, they're mixed case. So, we actually what we then have to do is come over to our resolvers. So, let's come to our resolver. And you actually have to add a resolver that translates these like that. And now when I save it, it's actually going to replace capital EUROPA with lowercase Europa, which now, in theory, it's kind of your back to what you want. Now, I've got a list, it pulls back the right thing, but now we're still disconnected a little bit. When I modify this session right here, I have to now change EUROPA to all capitals. And that's just a little bit of a disconnect, and it feels a little weird. Now, according to Apollo, where they recommend this is if you have a back-end API that you're trying to kind of wrap a façade over, and you want to clean it up a little bit and make it a little bit more nice and easy to use, that's a good place where enums can kind of come in where you should be passing code and you want to give clear names or things like that. Enums, on their surface, look cool, and you could potentially make use of them. They're not something I reach for ever, mostly just because I don't want to change code to change the list of options I have, and I don't necessarily like this weird translation from all caps to lowercase. But that's enums in a nutshell. All right, awesome. Let's take just a minute and summarize where we are and how we got here.

Summary



All right, we've come quite a ways in this module where we learned how to modify data. So, we can't just do queries all the time. We've got to learn how to modify data and change some things. And so, in this module, we talked about mutations, and we implemented `toggleFavoriteSession`, so the ability to update data. And we built a mutation, we added a resolver for that mutation, and we kind of worked through that whole process. We also added a new session, so we talked about how to create a new thing. And in covering that, you kind of get the basics of everything is very modularized. It's just a method that is enabled via an endpoint that enables you to change things in your data. We also talked real briefly about enums, and we talked about why I don't really like them, but what they are and how it enables you to list a set of options for any given type on input. All right, as we've seen a couple of times throughout the course of this course, things break. Errors happen. And so, what I want to do in this next module is show you how to handle errors in our code.

Error Handling and Validation

Introduction

Let's talk about how to deal with errors and when things break in Apollo and GraphQL. So, as we've seen throughout this course, things break. They just do. We hit errors. We type things wrong. Things happen. And we've seen that Apollo actually helps a lot. We've seen error messages come back in our API that we didn't right. And so, Apollo deals with some of that stuff for us right out of the box. But sometimes, the error message isn't exactly what we would like to see, and so I'm going to show you how to customize errors. We'll use `formatError`. We'll customize our errors and make them look maybe a little bit nicer. I'll also show you how to throw our own errors. So, sometimes errors happen that Apollo doesn't catch. And so, we'll talk about how to build our own errors and throw those instead. And then we'll end with this idea of unions where we can return back either the item that was requested or an error message. And so, we can kind of talk through what that looks like to make things maybe a little bit cleaner on the front end. All right, so let's talk about some built-in error handling that Apollo does for us automatically.

Built-in Error Handling

All right, now, Apollo has some built-in error handling that we can use to just basically make our lives easier. We don't have to deal with schema validation. We don't have to deal with a lot of that stuff cause Apollo is going to do that for us out of the box. So, real quick, let's look at what that looks like. So, let's say we send `time` in `sessionById`. `Time` doesn't exist, but we are requesting `time`. Well, guess



what? I'm going to get an error that says you can't query time just automatically out of the box. Another option for this is, let's say, room. Room is what we call a scalar type, and so it doesn't have any object. It's just a string. So, I can't query room name, which you might think— You might think room's an object and so it has a name. Well, that's not how that works. And so Apollo is going to say, hey, room doesn't have subfields. It's a type string. It's its own thing. And so, it builds these errors for us automatically out of the box now. Now, if something horrible happens— Let's say we kill our our Speakers API. But then, I still am querying speakers, so I still have to go get speakers here. It sends a message automatically, request to this speakers thing failed. That's because the data source threw an error because it couldn't hit the endpoint, and now we have this error. And we didn't write this error. We didn't throw this error. This error just happened, and it's gross, and we don't necessarily want to expose this to our end user. So, now that we know kind of what Apollo does and how it works through errors and it does its own validation, let's look at how to clean this up a little bit so we don't have to expose how gross this stuff is out to the end user.

Cleaner Error Messages

All right, so let's spend just a minute talking about how to clean up our error messages and just make those look a little bit nicer than what we get just out of the box for Apollo. So, let's start with this. Let's go back and do time. Let's add that in there, and we'll get an error that says cannot query field time, and that's exactly what we'd expect. And first things first, notice this stack trace here. That's a big no-no, like don't throw stack traces back to the end user. And the way we're going to kind of swallow that is, if we come in here to index.js and just set debug to false, you also can set production as your environment variable like we did earlier in the course. That's another option here. Now, when I run this, notice, most of that stuff goes away. So, that's a little bit nicer, a little bit cleaner. And so, it gets us on a start. It still says cannot query field time on type session. So, this is actually a nice, helpful error message. But let's say we want to swallow this. Or better yet, let's do this again where we turn off our speakers and we run. Oh, let's pull time back off because that doesn't exist, and then we run. All right, we get this really gross, big error. And notice the code, `INTERNAL_SERVER_ERROR`. So, let's talk about how to make this a little bit cleaner and a little bit nicer. We're going to take this in a couple of stages, and I'll show you a couple of different ways to do it. The first way we're going to look at is if we come in here to our index.js, in our apollo-server constructor, this takes a method called `formatError`, and it's actually going to send us the error and then allow us to do some stuff with it. And so, let's pick something. So, let's just say, hey, if `error.extensions.code = INTERNAL_SERVER_ERROR`, then we're going to return a new `ApolloError`. Now, `ApolloError` is pulled in up here at the top so we can use that, and we're just going to return a new `ApolloError` that has a message and a code. So, the message can be We are having some trouble, and the code can



be just ERROR. And then, actually, you can send extensions, just like we did on the other one. But maybe a nice, clean way to do this would be to send in a token and then maybe log this error to a database with a token. And then, when a user opens a ticket or something like that happens, you can look up that unique token and pull the error back out of the database. Okay, so let's save that. Now, let's see what we get when we run this. Ah, see that is much nicer and cleaner. And I also want to show you we still have our sessionByld down here. But notice, our speakers is null. So, it still ran the rest of it. It just swallowed up the error just like that and then continued to proceed and show the rest of the data, so we still have our sessionsByld and our sessions. It just so happens that we had speaker = null. So, that's kind of how to clean up your errors and make them look a little bit nicer. Let's maybe look in a different way next to actually preempt that a little bit and actually throw the error from inside our application when we notice that there's a problem down in there.

Throwing Errors

So, we just spent some time looking at how to deal with errors kind of globally. So, when when Apollo throws an error, we looked at formatError to kind of globally assess and clean up that ApolloError. What I want to do in this clip is take a more fine-grained approach and actually specifically throwing errors right where the error potentially would happened. So, if we look back at our code here, the specific scenario is speakers is turned off. So, that Speakers API isn't working at all. And so when we run it, it's throwing an error for speaker because it can't get to the Speaker API. And so, we've got a formatError that cleans that up. But that's kind of a hammer approach. It's just going to stomp on everything. What I want to do is actually come into the session resolver and actually deal with that error right here in this spot. So, if this Speakers API call throws an error, then we can actually catch that error and just swallow it. So, we could actually do a try-catch block here. Put everything inside the try and then catch return null. Now, with that change, when I run this, now I get speakers is null, speakers doesn't work, but the error went away. It just swallowed the error as a whole. So, we don't necessarily want that to happen. We want to construct a meaningful error at this moment that will be applicable and meaningful to us as we work through. And so, we're going to do the same thing we kind of did over here in index. We are going to use this ApolloError that we have and be specific. So, right here, we can say return ApolloError, and little cool little note for Visual Studio Code users. I don't know if you noticed what just happened, but that was super awesome. I'm going to Ctrl+Z. And then, since I have it over here in index, it kind of knows that ApolloError goes from here. When I return a new ApolloError, Visual Studio Code automatically sees that and pops it in up here at the top. That's kind of neat. All right, so what we want it to say is kind of a message of unable to retrieve speakers. And our code is going to be SPEAKERAPIERROR. And then, again, I kind of like the token approach kind of like we did over here where we actually send a



token to the error that then we log in a database somewhere, and then you can go look up and see what the problem is. So, now, we'll get this cool new `ApolloError` and not the big hammer error of we're having a problem. Well, we got the errors of null, but it swallowed the whole thing up. So, let's think about why that is for just a second and kind of work through this. We forgot to do something in the last clip that I'll kind of show you. So, here, we're throwing a new `ApolloError`. But `formatError` is getting called because everything's going to flow through that `formatError`. And here we're saying, hey, if `INTERNAL_SERVER_ERROR`, return new `ApolloError` right there. But we don't actually ever like return the rest of the errors. So, let's do that. Boom. There you go. I'm able to retrieve speakers, all of that. It's all clean. It's all good to go. Now, you may not like this locations thing, and that's kind of like locations or path, or that's the kind of stuff that can be cleaned up in `formatError`. You can just modify these error messages as you want and pull things in and put them out however you want to deal with that. Now, this still breaks down to notice we have errors, and we have data. And so, you've got to check errors and data. There's one more option here for us, and that's to actually use unions, what we haven't talked about yet. But unions that have the option to return either data or an error. And that way, we can dig in a little bit better and maybe provide a different approach that may work better for what you're trying to do.

Unions

All right, so what I want to do now is kind of come up with this idea of unions being used to make our errors potentially a little cleaner. And it's going to be a personal style thing. That's why I've showed you all the different options that we have. But this is one option that makes things maybe a little bit cleaner depending on what your personal preferences are. So, if you come into our schema and you look at `sessionById`. And so, right there it returns a `Session`. And if an error happens, then it gets caught up over an index in our `formatError`, and we throw this new `ApolloError`, and everything just kind of works. But sometimes, maybe we want things to be a little bit nicer and cleaner and maybe actually just return a type, actually create a type that we control of error. And that error is going to have a code, maybe a message, and then that token thing that I keep referencing. There you go. Now we have a type. And then I want my `sessionById`, this thing that we're returning, to actually return a session or an error, just like that. And so, if an error happened, it's going to return an error. If a session happened, it's going to return a session. And that's how that works. And the way we make this work is we actually create a thing called a union. And so, what a union is is exactly what it sounds. It's a combination of two types. And so, we can create a union of session or error that is basically `Session` unioned with `Error`, just like that. Now, there's a couple things we need to do to kind of make this work. And one thing we want to do is actually resolve the type. And so, because really, we're going to send back either a session or an



error, and I want to give the client the idea that it is a session or an error depending on what happened. And so, what you can do is you can actually come into our resolver and create a resolver for `SessionOrError` that uses an internal thing called `resolveType`. Now, this is a method that gets called by Apollo to resolve the type on this object. And depending on whether an error happened or not, this is either going to be a session or it's going to be an error. It's not really going to be a combination of the two things because either one happened or the other. So, they're going to pass as an object, and we get to decide what it is. It's either one or the other. And so, we can actually say, hey, if `obj.code`. So, if there's a code on this, then it is an `Error`. Otherwise, it's a `Session`. That's how that works. And so, now, I'm returning the appropriate type, and so that should work. Let's actually look at our code. Yep, so you even get an error. So, when I saved it one time look here, `SessionOrError` is missing a `resolveType` resolver. So, it actually is required, and you'll get a warning if you don't do it. But I'm up and running, so let's tab over to our browser, and let's run this. Notice, everything's broken. Of course it is. So, refresh, and we'll actually get the red over here on the side because `sessionByld` if we look at our schema, `sessionByld` is returning a `SessionOrError`. It's not returning a `Session`. And so, `title` is actually not on `SessionOrError`. It's on `Session`. And `favorite` is not on `SessionOrError`. But then, if I've returned an error, I don't have in this `sessionByld` query that I'm pulling the error stuff back. And so, what we actually have to do here is use what we call fragment notation. So, let's dot, dot, dot. So, on `Session`, we want to pull all of that stuff. But on `Error`, we want to pull the code, the message, and the token. So, now, let's run this. What's going to happen? Oh, look. It worked exactly the way we wanted it to. But what happens if I throw an error? What happens if something horrible happens? So, let's come over back to our code and say, hey, just in our data source, let's have `getSessionsByld`, let's just throw, just like that. And we'll save it. Now, when we run it, ah. I get the global `We are having some trouble`. So, `getSessionByld` is now broken. I can't use that. So, if I come into my resolver, my query resolver, and I go to `sessionByld`, let's wrap this in a try-catch block. So, we'll try this. Otherwise, we return an error object, so add a code of `ERROR`. Let's just make this simple. We'll just hardcode this for right now. An error occurred. And then, our token. Okay, so now, if an error happens, we catch the error, and then we just return an object. We're not throwing anything anymore. We're not going through that whole deal. We're just returning an object. And so, now, when I run this, notice `sessionByld` is just an error code. It's just clean. Everything's in data. I don't have an error section up above. It's just clean and concise and self-contained right there. So, I'm not saying this is the right way. I showed you three different ways to handle errors because some of it's personal preference, and it depends on what your requirements are and what you're trying to accomplish. And so, this is another way to kind of eliminate the error section and give you an option to return back an error just embedded within the object itself. So, let's take a minute and summarize what we've got, and then we'll do some conversation around API management in the next module.



Summary

All right, in this module, we talked about how things will break, and we looked at how errors can be handled in several different ways as we worked through this module. And we started with this conversation about how Apollo does a lot of error handling for us. It does the schema validation for us. It just handles some of those more complicated things, just kind of out of the box. But the errors there a little bit cumbersome or not necessarily sanitized in a way that's going to be good for the client application to consume. And so, I showed you how to use `formatError` and kind of how to customize those errors so they look a little bit nicer and cleaner. Then, we talked about how to throw our own errors, how to use try-catch block and swallow an error and then throw a new `ApolloError` and do those types of things in order to swallow errors right there in the moment and then do what we need them at a fine-grained level instead of just at the holistic level. And then we ended with a conversation about unions and how you can, instead of throwing errors and returning an error object, use unions in order to sometimes have an error embedded within the object itself that you're returning. All right, so that's it for the meat of the course, actually. So we've talked about a lot of different things as we went through there. One thing I want to do though is give you a look at Apollo Studio. We're not going to go deep into Apollo Studio, but I at least want you to see this option that Apollo has for you to help you deal with your GraphQL schema at kind of an enterprise level.

Working with Apollo Studio

Introduction

All right, now that we have a fully functional API, we've got speakers, we've got sessions, we've got some error handling, we're making changes, all of that stuff is there, I want to take a minute just to kind of talk you through a tool that Apollo has called Apollo Studio that might help you manage your API maybe a little bit easier. So, on top of having an open-source GraphQL solution for a client server, iOS, all of that, they also have a tool for monitoring your graph and tracking changes in and making sure everything is running smoothly. So, the first thing I want to do kind of in this module is I'm going to show you how to wire up your graph into Apollo Studio so you can get a look at what it contains, what your schema looks like, all of those types of things in the Apollo Studio tool. Then, we're going to turn on monitoring. I'll show you how to do error tracking. I'll show you how to do usage statistics, all of that bundled into Apollo Studio so you can understand what's going on as people use your API. And then finally, we'll talk about tracking changes. As your schema changes, getting an understanding of what's changing from version to version is important. And so, you can do that specifically with Apollo Studio and track the versions and make sure that you



aren't breaking things from one step to the next. All right, so real quick, let's dive into Apollo Studio. Let's look at what the tool is and then wire our API up to it as we go.

Wiring up Your API

All right, now the first thing we're going to need to do in order to explore Apollo Studio is actually like wire the API up to Apollo Studio. So, let's start with that, that basic first step in order to get things all hooked up together. All right, so if you run out to studio.apollographql.com and create your account and get all that stuff going, I'm going to create a new organization. We're going to create an organization called PSSessionAPI. Now, the organization ID must be globally unique. So, you can't use the same one I'm using. Come up with your own, and then work your way through that. All right, now, look at the plans here. So, you've got the free plan, which lets you do a lot of the basic stuff. And then, you've got the Team plan, which is what you would do if you had a big development team and all of that kind of stuff going on. But here, look, you've got alerts, you've got trace inspection, you've got performance metrics, all of those types of things built in. And so, actually, what I'm going to do just for this is I'm going to go try free for 14 days. We'll try this out. There you go. We're not going to invite any team members. Okay, now here we are. We've got a graph, an organization called PSSessionAPI and our graph. Now, we don't have any graphs yet. So, create your first graph, and this is kind of one of those lingo things. We haven't really talked about this. A graph would be the API itself, the whole big thing. So, what we want to do is create a new graph, and then this is going to be the sessionAPI. All right, so the cool thing about this is it will wire up directly to your server. So, there's a couple of different ways you can do this. You can embed something into your environment. I don't want to do that. What I want to do is use the command line. And so, if we have a running server, which we do right now, if you've still got yours running, great. If you don't start it back up again. But you can use the Apollo CLI to actually push your server up to Apollo Studio. Let's copy this code, and I'll talk you through that. Okay, so what we're going to do is we're just going to paste this, and I want to talk you through a couple of things. First and foremost, npx. If you are not familiar with npx— Normally, if I wanted to run the Apollo CLI, I would do an `npm install -g`, so that global install thing. Npx, what it does is say I'm not going to do the `-g`. I can run `npx apollo`, and it downloads it and runs it all in one command. So, you no longer really have to do that `npm install -g` if you're doing any of the CLI tools. You just run npx and that. And then, we are pushing our sessionAPI. There's our user and all of that stuff. And then, our endpoint is localhost 4000 because that's what we're running locally. If you had a server, a dev server or a prod server or whatever, you would obviously put that endpoint there instead of your localhost 4000. You're going to hit Run. Now I want you to see, look, watch behind me. There you go. It now picked up my API, and now this is wired up. So, I have my sessionAPI right here. And no



requests. Nothing has happened to it yet. And the last schema was published just a few seconds ago, obviously. Yay! And then you get all kinds of insights into what this API looks like. Mostly, you've got your fields. There's your mutations, your queries, your errors, your sessions, all of that stuff. This is now wired up. Now, I'm not going to go through any more of this yet because we're going to do that in the next clip. But once you get to this point, you are wired up, ready to go. And now we can start monitoring our API and seeing what's going on behind the scenes.

Monitoring Your API

All right, we've got our API wired up into Apollo Studio. Now, let's see what it takes to monitor our API and see what's going on. All right, where we left off, we've got our fields displayed. Everything's wired up and good to go. If we go down to monitor the operations, we want to set up our Apollo Server for real-time analytics. And so, there's two things we need to do. We need to add an environment variable, and then we need to turn on the graph variant into our API. So, here we go. Let's start with this `APOLLO_KEY`. Now, the easiest way for us to add this stuff into our environment is we're going to come into our `package.json` file. And we're using `nodemon`. So, start `nodemon index`, which means we can create `nodemon` environmental variables, so `nodemon config`, don't forget this is a JSON document so quotes are necessary, `env APOLLO_KEY` like that. All right, let's save that. But `Ctrl+C` since we changed the `nodemon` config, you actually have to stop and restart. But before we do that, let's go into `index`, and we have to pull this engine line in. So, right under our `formatError`, we'll just pull that right there. Save that. Okay, start this back up. Now, we don't have any metrics yet. Let's go `localhost 4000` and then run this, and we get the error occurred. And remember the error occurred because we broke `sessionById` fundamentally in the last clip. But if we go back to Apollo Studio, now, look at this. We've got our performance stuff in here, errors, traces, operations. Now, it says no errors. And the reason why it says no errors is because we swallowed it. Let's go into our resolvers, our `query.js`, and actually take this out. When I say we swallowed it, instead of throwing an error here in `index`, we actually returned an error that doesn't really count for an error. We'll save that. Now, I actually get a real error. And so, let's come back over to `sessionAPI`, and now you see here's our request rate, our service time, our error percentages, all of that stuff right here. And actually, the cool part about it is if you come over here into Errors, you can actually see the operation and the error that was associated with it and all of that right there. You can dig into specific operations that happened and so on. So, this is it. You get real-time performance metrics right here on Apollo Studio as your clients work through using the API. Now, you can filter by clients, all of that. This is actually really powerful to really dig in and understand some things that are going on. And that's just a real quick look at how to get that set up in your application. Now, the last thing I'm going to show you in Apollo Studio is how



to version your API in Apollo Studio.

Tracking Versions

All right, the last thing I want to do is show you this piece in Apollo Studio that lets you track the versions of your API because things will change over time. You want to make changes to your API, you want to deprecate things, you want to add new things. And so, Apollo Studio helps understand what's going on inside your API schema as you work. Let's look at what that looks like. Okay, so let's go to our `schema.js`, and let's actually take out— Let's just drop— We said we were going to deprecate `track`. We're deprecating it. So, now, let's remove it. Let's save that. Notice that'll run. And then, we're going to go in here to History. And notice, we have our initial schema. We haven't published a new schema yet because it doesn't pick that up automatically. So, what you normally would do is as part of your CI/CD process as you're deploying, you run your `npx` again, and we're going to push a new version of our API. Notice this. This is red, broken query. And the reason why it does, this is a part of Apollo Studio that I wanted to call out very specifically because it says, hey, you removed right here `track`. Session: field `track` was removed. And there are operations that are executing against your API that actively use that field. And so it's going to throw up in error and say, hey, did you mean to do this because you just broke a bunch of your client operation. And so that's the thing that makes this interesting and gives you some tools to make some more intelligent decisions about what's going on inside your schema. So, that is a very quick little look at Apollo Studio. I'm not going to dig into all parts of it. I just want to give you just some exposure to understand what's going on with Apollo Studio and how it works so that now you can go explore and see if it's something that might make sense for you and your specific environment. So, that's it. Let's take a minute to summarize kind of where we are and what we've done in this module.

Summary

Wow, we've come quite a way in the course of this course over the last several modules. In this last module, we focused on Apollo for monitoring. So, we looked at Apollo Studio and using that to monitor and keeping an eye on your API. We started by wiring up our graph, actually creating an organization in Apollo Studio, then using the Apollo CLI to wire up your API out to Apollo Studio. And then we used that to monitor our API. I showed you how to look at performance and errors and just kind of get a quick glimpse of what's happening inside your API. And then the cool part about it is it knows what things are being executed against your API so that when changes happen, when you change your schema, it will let you know if something is going to break the things that are actively being run against your API. So, that's kind of a huge value add that you can get out of



Apollo Studio if you do that. Well, that's it. We've made it all the way through this course. And so, thank you very much for sticking with me. But we have created an API. We created queries. We created mutations. We used REST services. We did all of that stuff all bundled in. And now you have all the tools you need to go out on your own and get started building a GraphQL API with Apollo Server.